

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО

Домашняя работа №5
по дисциплине
"Web-разработка: Frontend"

Семестр IV

Выполнил:
студент
Бровкин Аким Алексеевич
гр. J3210
ИСУ 465282

Санкт-Петербург
2026

1 Цель работы

Целью данной работы является изучение основ разработки одностраничных веб-приложений с использованием фреймворка Vue.js 3-й версии.

2 Задачи работы

1. Инициализировать проект с помощью Vue CLI и настроить базовое окружение.
2. Разработать компонентную архитектуру приложения с использованием библиотеки Bootstrap для стилизации.
3. Настроить маршрутизацию приложения посредством Vue Router.
4. Реализовать управление централизованным состоянием приложения с помощью хранилища Pinia.

3 Описание реализации

3.1 Архитектура приложения и компоненты

В ходе работы я разделил интерфейс приложения на переиспользуемые компоненты. Был создан базовый шаблон `BaseLayout.vue`, который включает в себя навигационную панель и подвал, а также тег `<router-view>` для динамической подмены контента страницы.

Для отображения отдельной записи я разработал дочерний компонент `NoteCard.vue`, который принимает данные (заголовок и текст) через `props` и отправляет событие удаления наверх с помощью `$emit`.

3.2 Управление состоянием и роутинг

Для обеспечения корректной работы приложения без перезагрузки страниц я внедрил `vue-router`. Основная логика работы с данными была вынесена из компонентов в изолированное хранилище `notes.js` с использованием `Pinia`. Это позволило централизованно управлять массивом заметок, добавлять новые записи и удалять существующие.

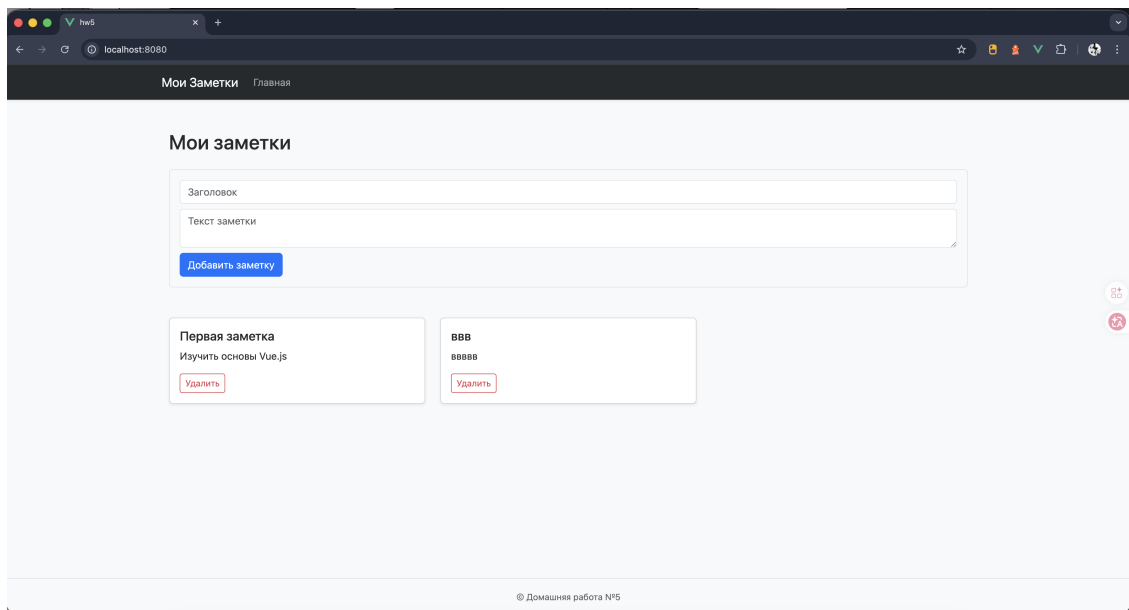


Рис. 1: Пользовательский интерфейс приложения

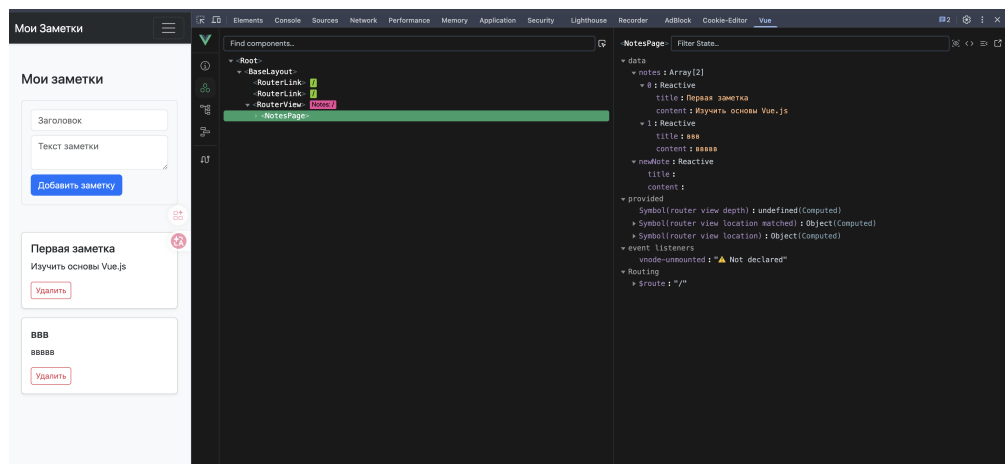


Рис. 2: Отображение состояния в Vue DevTools

4 Примеры реализации

Ниже приведен исходный код основных разработанных компонентов приложения:

Листинг 1: Компонент карточки заметки

```
<template>
  <div class="card mb-3 shadow-sm">
    <div class="card-body">
      <h5 class="card-title">{{ title }}</h5>
      <p class="card-text">{{ content }}</p>
      <button class="btn btn-outline-danger btn-sm" @click="$emit('
        delete-note')">

      </button>
    </div>
  </div>
</template>
```

```

<script>
export default {
  name: 'NoteCard',
  props: {
    title: String,
    content: String
  },
  emits: ['delete-note']
}
</script>

```

Листинг 2: Хранилище состояния

```

import { defineStore } from 'pinia'

export const useNotesStore = defineStore('notes', {
  state: () => ({
    notes: [
      { id: 1, title: '
                                Vue.js' }, content: '
    ]
  })),
  actions: {
    addNote(note) {
      this.notes.push({
        id: Date.now(),
        ...note
      })
    },
    removeNote(id) {
      this.notes = this.notes.filter(note => note.id !== id)
    }
  }
})

```

5 Заключение

В ходе выполнения домашней работы я на практике ознакомился с принципами компонентного подхода во Vue.js. Мною были успешно интегрированы инструменты маршрутизации и управления состоянием. Созданное SPA-приложение позволяет корректно обрабатывать пользовательский ввод и динамически обновлять интерфейс.